JAVASCRIPT • MODULE 07

ES6+ Features

Modern JavaScript You Must Know

■ MODERN JS

■ LEARNING OUTCOME

By the end of this module you will use all ES6+ features fluently: modules, classes, generators, Symbols, WeakMap, WeakSet, Proxy, and the latest proposals.

■ Skills You Will Gain

- Write and import ES6 modules with named and default exports
- Create classes with inheritance, getters/setters, private fields
- Use Symbol for unique keys
- Understand generators and iterators
- Use Proxy for validation and reactivity
- Apply WeakMap and WeakSet

■ ES6+ is not optional -- every modern codebase, framework, and interview uses these features daily. React, Vue, Angular, and Node.js are all built with ES6+ patterns.

SECTION 1

ES6 Modules and Classes

```
// ====== MODULES ====== // math.js -- named exports export const PI = 3.14159; export function add(a,b) { return a+b; } export function subtract(a,b) { return a-b; } // UserService.js -- default export export default class UserService { constructor(apiUrl) { this.apiUrl = apiUrl; } async getUser(id) { /* ... */ } } // IMPORTING import { PI, add, subtract } from './math.js'; // named import { add as sum } from './math.js'; // rename import * as MathUtils from './math.js'; // all as namespace import UserService from './UserService.js'; // default import UserService, { PI } from './UserService.js'; // mixed // Dynamic import (lazy loading) const { default: Chart } = await import('./Chart.js'); // ====== CLASSES ====== class Animal { #name; // private field species = 'Unknown'; // public field constructor(name, sound) { this.#name = name; // private this.sound = sound; // public } speak() { return `${this.#name} says ${this.sound}!`; } get name() { return this.#name; } set name(value) { if (!value) throw new TypeError('Name required'); this.#name = value; } static create(name,sound) { return new Animal(name,sound); } } // INHERITANCE class Dog extends Animal { #tricks = []; constructor(name) { super(name, 'Woof'); // MUST call super() first! } learn(trick) { this.#tricks.push(trick); return this; } perform() { return `${this.name}: ${this.#tricks.join(', ')}`; } speak() { return super.speak() + ' Good dog!'; } // override + super } const dog = new Dog('Rex'); dog.learn('sit').learn('roll over'); // fluent chaining console.log(dog instanceof Animal); // true
```

SECTION 2

Symbols, Generators, Proxy

```
// SYMBOLS -- guaranteed unique values const id1 = Symbol('id'); const id2 = Symbol('id'); id1 === id2 // false -- always unique! // Well-known Symbol: make class iterable class Range { constructor(start, end) { this.start=start; this.end=end; } [Symbol.iterator]() { let current = this.start, end = this.end; return { next() { if (current <= end) return { value: current++, done: false }; return { done: true }; } }; } } for (const n of new Range(1,5)) console.log(n); // 1 2 3 4 5 const nums = [...new Range(1,5)]; // [1,2,3,4,5] // GENERATORS -- pausable functions function* fibonacci() { let [a,b] = [0,1]; while (true) { yield a; [a,b]=[b,a+b]; } } const fib = fibonacci(); fib.next().value // 0 fib.next().value // 1 fib.next().value // 1 fib.next().value // 2 // PROXY -- intercept object operations const validator = new Proxy({}, { set(target, prop, value) { if (prop === 'age' && (typeof value !== 'number' || value < 0)) { throw new TypeError(`Invalid age: ${value}`); } return Reflect.set(target, prop, value); } }); validator.age = 25; // OK // validator.age = -1; // TypeError! // Proxy for reactive state function reactive(obj, onChange) { return new Proxy(obj, { set(target, prop, value) { const old = target[prop]; Reflect.set(target, prop, value); onChange(prop, old, value); return true; } }); } const state = reactive({ count: 0 }, (p,o,n) => updateDOM(p,n)); state.count = 5; // automatically updates DOM!
```

■ PRACTICE Q & A

Q1. What is the difference between named and default exports?

> ***A:*** *Named: export {name,fn} -- import with exact name in {}. Multiple per file. Default: export default X -- import with any name, no {}. Best practice: named for utilities, default for main component per file.*

Q2. What makes a private class field (#field) different from a regular property?

> ***A:*** *Private fields are truly private -- only accessible inside the class body, not from outside or subclasses. Regular properties are public. Private fields require # prefix and exist in modern JS (ES2022+).*

Q3. What is a generator function and what does yield do?

> ***A:*** *A generator (function*) is a pausable function. yield pauses execution and returns a value. Resumes when .next() is called. Creates lazy, memory-efficient infinite sequences.*

Q4. What is a Proxy used for?

A: *Wraps an object and intercepts operations (get, set, delete). Used for: input validation, reactive state (Vue 3), access control, logging, API mocking.*

Q5. What is Symbol.iterator?

A: *A well-known Symbol that makes objects iterable -- usable in for...of, spread, destructuring. Implementing [Symbol.iterator]() on your class makes it work with all iteration protocols.*

■ ES6+ Cheat Sheet	
`export { fn1, fn2 }`	Named exports
`export default Component`	Default export
`import { fn } from './mod'`	Named import
`import Component from './mod'`	Default import
`import('./module')`	Dynamic lazy import
`class Child extends Parent {}`	Inheritance
`super(args)`	Call parent constructor
`#privateField`	True private class field
`get/set propName()`	Getter and setter
`static method()`	Class-level static method
`Symbol('desc')`	Unique identifier
`function* gen(){ yield v; }`	Generator function